Facultad de Ingeniería
Escuela de Ingeniería Informática

Implementación de una plataforma centralizada de gestión, correlación y priorización de vulnerabilidades integrada al SOC

Jorge Eduardo Álvarez Alarcón

Trabajo realizado para optar al título de
Ingeniero Civil Informático

Prof. Guía: **Cristian Carrión**

Valparaíso, Junio 2026

Certificación

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Cristian Carrión
Profesor Guía

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Nombre Profesor Informante 1
Profesor Informante

Certifico que he leído este documento y que, en mi opinión, es adecuado en ámbito y calidad como trabajo para optar al título de Ingeniero Civil Informático.

Nombre Profesor Informante 2
Profesor Informante

Aprobado por la Escuela de Ingeniería Informática, Universidad de Valparaíso.

Declaración sobre el uso de herramientas de Inteligencia Artificial

Yo, **Jorge Eduardo Álvarez Alarcón**, autor del trabajo "*Implementación de una plataforma centralizada de gestión, correlación y priorización de vulnerabilidades integrada al SOC*", declaro que el contenido de este documento es original y ha sido elaborado principalmente mediante mi propio esfuerzo intelectual, en conformidad con los principios de integridad académica exigidos por la universidad.

Declaro asimismo que se han utilizado herramientas de Inteligencia Artificial, tales como *modelos de lenguaje de gran escala para asistencia de texto*, exclusivamente como apoyo en tareas auxiliares de redacción, mejora de estilo, corrección gramatical y *revisión de coherencia sintáctica en la estructuración de párrafos*. Estas herramientas no han sido empleadas para la generación autónoma de contribuciones sustantivas del trabajo, tales como el planteamiento del problema, el diseño metodológico, la implementación ni el análisis de resultados, los cuales corresponden íntegramente al autor.

Todo contenido asistido por IA ha sido revisado, validado y adaptado críticamente, garantizando su pertinencia, exactitud y originalidad. Asumo plena responsabilidad por el contenido final del documento.

Se deja constancia de que el uso de estas herramientas ha sido realizado con conocimiento de **Cristian Carrión**, profesor/a guía, en conformidad con las normativas institucionales y buenas prácticas académicas vigentes.

Implementación de una plataforma centralizada de gestión, correlación y priorización de vulnerabilidades integrada al SOC

Jorge Eduardo Álvarez Alarcón[a,*], Cristian Carrión[a]

[a]*Escuela de Ingeniería Informática, Universidad de Valparaíso, Valparaíso, Chile*

Resumen

Este documento presenta el diseño e implementación de una plataforma enfocada en resolver la problemática de la fatiga de alertas y la ineficiencia en el triaje dentro de los Centros de Operaciones de Seguridad (SOC). Actualmente, la fragmentación de herramientas de defensa y el uso de métricas de evaluación estáticas limitan la capacidad operativa de los analistas, generando cuellos de botella en la respuesta a incidentes. Para abordar este desafío, se propone una plataforma centralizada bajo una arquitectura SOAR (Security Orchestration, Automation, and Response) y un enfoque de Gestión de Vulnerabilidades Basada en Riesgo (RBVM). Mediante la correlación automatizada de alertas del SOC con fuentes dinámicas de Inteligencia de Amenazas, la solución prioriza riesgos reales, filtra falsos positivos y facilita la remediación. El presente trabajo detalla la fundamentación teórica, la arquitectura de integración de la solución y la definición del estudio experimental para validar la eficacia del modelo propuesto en la reducción de tiempos de respuesta.

Palabras clave: SOC, SOAR, RBVM, Fatiga de alertas, Ciberseguridad, Triaje

Metadata del software

Tabla 1: Información general de la plataforma SOAR

Descripción	Metadata
Arquitectura	Integración mediante APIs (Wazuh, OpenVAS, MISP)
Motor Lógico (Backend)	Python 3.x
Base de Datos	MongoDB (NoSQL)
Interfaz (Frontend)	React.js
Entorno de ejecución	Contenedores Docker efímeros y ejecución nativa local

*Contacto: Jorge Eduardo Álvarez Alarcón (jorge.alvareza@estudiantes.uv.cl)

1. Introducción

En el escenario actual de ciberseguridad, los Centros de Operaciones de Seguridad (SOC) se han constituido como infraestructuras críticas donde equipos de profesionales monitorean, detectan y responden a incidentes apoyados por tecnologías avanzadas [1]. La efectividad de estas unidades es vital para la resiliencia de las organizaciones ante amenazas que evolucionan rápidamente en complejidad y escala, tales como ataques de día cero y ransomware.

Sin embargo, la proliferación de herramientas de defensa ha generado una saturación de datos que los analistas deben procesar manualmente. Este fenómeno, conocido como "fatiga de alertas", es causado por el alto volumen de notificaciones, una excesiva tasa de falsos positivos y la existencia de silos de información en tableros desconectados.

Como respuesta a esta problemática, el presente trabajo propone la implementación de una plataforma centralizada bajo una arquitectura de Orquestación, Automatización y Respuesta de Seguridad (SOAR). La principal contribución de este proyecto es la integración de fuentes de inteligencia de amenazas mediante un enfoque de gestión basado en riesgo (RBVM), automatizando la correlación de eventos para optimizar el triaje.

El resto de este documento se estructura de la siguiente manera: la Sección 2 presenta el marco conceptual que fundamenta el proyecto; la Sección 3 analiza el estado del arte y trabajos relacionados; la Sección 4 define la problemática y los objetivos; la Sección 5 detalla el diseño de la solución tecnológica propuesta; la Sección 6 define el estudio experimental para su validación; y finalmente, la Sección 7 establece las limitaciones del sistema.

2. Marco Conceptual

Para fundamentar el diseño del motor de orquestación y priorización, es imperativo establecer con precisión los pilares teóricos y tecnológicos que convergen en la solución propuesta. Esta sección define conceptualmente los entornos de monitoreo moderno, los paradigmas de automatización y los modelos analíticos de evaluación de riesgo en ciberseguridad.

2.1. Centros de Operaciones de Seguridad (SOC) y Sistemas SIEM

Un Centro de Operaciones de Seguridad (SOC) se define como una unidad centralizada e infraestructura sociotécnica encargada de supervisar, evaluar y defender la postura de seguridad de una organización en tiempo real [1]. El componente tecnológico central sobre el cual se articula un SOC tradicional es el sistema de Gestión de Eventos e Información de Seguridad (SIEM, por sus siglas en inglés).

Los sistemas SIEM cumplen funciones analíticas críticas organizadas en dos dimensiones operativas:

- **Gestión de Información de Seguridad (SIM):** Encargada de la recolección centralizada, normalización, almacenamiento a largo plazo y parsing de registros (*logs*) generados por dispositivos perimetrales, servidores, aplicaciones y *endpoints*.
- **Gestión de Eventos de Seguridad (SEM):** Provee capacidades de análisis correlacional y monitoreo en tiempo real, aplicando reglas determinísticas sobre el flujo de eventos entrantes para identificar actividades anómalas o patrones de ataque conocidos.

En infraestructuras de código abierto, soluciones distribuidas basadas en agentes como *Wazuh* ejecutan estas funciones combinando el análisis sintáctico de telemetría de *host* (HIDS) y de red (NIDS) para establecer la visibilidad basal y proveer la evidencia operativa del entorno operacional [2].

2.2. Orquestación, Automatización y Respuesta de Seguridad (SOAR)

Las limitaciones intrínsecas de los SIEM para responder de forma autónoma impulsaron el desarrollo de las plataformas de Orquestación, Automatización y Respuesta de Seguridad (SOAR). Mientras que el SIEM se enfoca en la agregación y visibilidad de eventos, el paradigma SOAR se orienta a la interoperabilidad de herramientas aisladas y la automatización del flujo de trabajo informático [3].

Un ecosistema SOAR se compone formalmente de tres capacidades núcleo:

1. **Orquestación:** Capacidad de integrar herramientas de seguridad heterogéneas (SIEM, escáneres de vulnerabilidades, repositorios de inteligencia) mediante abstracciones de software y APIs REST, consolidando silos tecnológicos en un plano de control unificado.

2. **Automatización:** Ejecución de tareas técnicas programadas a través de flujos de trabajo lógicos denominados *playbooks* o guías de respuesta, reduciendo los tiempos de intervención manual frente a incidentes tipificados.

3. **Respuesta Guiada:** Centralización del triaje en interfaces dinámicas que enriquecen los datos basales con contexto organizativo, asistiendo activamente al analista humano en la toma de decisiones críticas.

2.3. Evaluación Tradicional de Vulnerabilidades (CVSS) y sus Limitaciones

El Sistema de Puntuación de Vulnerabilidades Comunes (CVSS) es el estándar de la industria administrado por el FIRST (*Forum of Incident Response and Security Teams*) para cuantificar la gravedad técnica intrínseca de una falla de seguridad informática detectada de forma pasiva o activa [4]. CVSS genera un índice numérico en un rango de 0.0 a 10.0 basado en tres grupos de métricas principales: métricas base (atributos inherentes a la vulnerabilidad), temporales (estado actual de explotación y parches disponibles) y ambientales (impacto específico dentro de la infraestructura local).

A pesar de su adopción universal, la implementación pura de la métrica base CVSS presenta severas limitaciones operativas en un SOC moderno:

- **Ausencia de Contexto Dinámico:** El cálculo base de la *National Vulnerability Database* (NVD) es inmutable y simétrico; evalúa el vector técnico de la vulnerabilidad sin considerar si el activo afectado es un servidor crítico expuesto a internet o una estación de trabajo aislada.

- **Saturación del Triaje:** La proliferación de fallas categorizadas de severidad Alta o Crítica (puntajes ≥ 7.0) genera una dispersión de recursos, induciendo directamente al fenómeno técnico de fatiga de alertas al carecer de un discriminador de explotación real activa [1, 4].

2.4. Gestión de Vulnerabilidades Basada en Riesgo (RBVM)

Para subsanar las ineficiencias del enfoque tradicional estático, surge el paradigma de Gestión de Vulnerabilidades Basada en Riesgo (RBVM). Este modelo transiciona la estrategia defensiva desde una postura de mitigación masiva basada exclusivamente en la severidad técnica del código, hacia un enfoque de optimización analítica guiado por el riesgo matemático [5].

Bajo el enfoque RBVM, el riesgo informático se modela formalmente como una función multidimensional dependiente de la amenaza real, la severidad intrínseca y el impacto específico del negocio [6]. Esto permite calcular una ordenación priorizada que optimiza los recursos de remediación, concentrando el esfuerzo operativo en la intersección donde una falla altamente severa afecta a un activo de alto valor estratégico y presenta vectores de ataque activos comprobables en la telemetría operativa.

2.5. Inteligencia de Amenazas (CTI) y Plataformas de Intercambio (MISP)

La Inteligencia de Amenazas Cibernéticas (CTI) comprende la recolección, procesamiento y análisis analítico de datos empíricos del mundo real respecto a las capacidades, vectores, motivaciones e identidades de actores maliciosos activos [7]. CTI transforma indicadores técnicos abstractos en conocimiento táctico accionable para robustecer la detección predictiva del SOC.

Para la ingesta y correlación eficiente de estos flujos masivos de datos sin penalizar la latencia operativa, se emplean plataformas especializadas de código abierto como MISP (*Malware Information Sharing Platform*) [8]. MISP actúa como un bus de integración y caché local de conocimiento táctico, permitiendo almacenar, estructurar bajo taxonomías normalizadas y consultar instantáneamente mediante llamadas programáticas locales Indicadores de Compromiso (IoCs) y repositorios consolidados externos, tales como el catálogo de Vulnerabilidades Explotadas Conocidas (CISA KEV) [9]. Esta consulta persistente en un entorno local optimiza el cálculo del índice dinámico de riesgo sin depender de conexiones directas y síncronas hacia servicios externos en internet durante el proceso crítico de triaje operacional.

3. Estado del Arte y Trabajos Relacionados

3.1. El Problema de la Fatiga y la Dependencia Estática

Mientras que el Marco Conceptual define la estructura teórica de un SOC y el estándar de medición CVSS, la literatura empírica reciente evidencia el colapso operativo de este modelo tradicional. Tariq et al. [1] diagnostican que la dependencia excesiva de los sistemas SIEM en reglas estáticas ha transformado la gestión de eventos en un problema sociotécnico; la "fatiga de alertas"desensibiliza a los analistas y genera falsos negativos críticos. Paralelamente, la academia critica la aplicación aislada del CVSS, demostrando que una métrica de severidad puramente teórica carece de valor defensivo si no se ajusta dinámicamente al contexto del producto afectado y la red local [4].

3.2. Evolución hacia la Priorización Contextual e Inteligencia

Para resolver esta brecha entre la teoría y la operación real, la investigación aplicada se ha volcado hacia el modelo de Gestión Basada en Riesgo (RBVM) y la Inteligencia de Amenazas (CTI). En entornos SOC, autores como Abakar et al. [7] y Kits et al. [8] han propuesto integrar plataformas como MISP para mantener repositorios locales de conocimiento táctico. El uso de herramientas de código abierto permite cruzar las vulnerabilidades detectadas con catálogos de explotación activa, como el CISA KEV [9]. Investigaciones contemporáneas ya intentan unificar

el CVSS con el CISA KEV y la criticidad de los activos corporativos para generar índices de priorización dinámica [6], validando la necesidad de un enfoque multicriterio.

3.3. Orquestación Comercial frente a Transparencia Local

A nivel arquitectónico, la industria ha consolidado el paradigma SOAR para automatizar el triaje [3], buscando complementar la ingesta masiva de telemetría proveniente de herramientas como Wazuh [2]. No obstante, el estado actual del mercado está dominado por soluciones comerciales alojadas en la nube (tales como Microsoft Sentinel o Splunk). Aunque estas plataformas logran mitigar la carga cognitiva utilizando sofisticados modelos de Inteligencia Artificial (IA), operan predominantemente como cajas negras algorítmicas. Este enfoque exige a las organizaciones delegar la telemetría sensible de su infraestructura a nubes de terceros y dificulta la auditoría matemática exacta de por qué un incidente fue escalado o ignorado por la IA.

3.4. Aporte y Diferenciación del Proyecto

El hilo conductor de la problemática actual exige una solución que sea dinámica para filtrar el ruido, pero auditable para garantizar la confiabilidad. A diferencia de las soluciones comerciales predictivas opacas, este trabajo de título propone un diseño RBVM local, determinístico y transparente [5]. La innovación principal radica en operacionalizar la teoría matemática mediante un motor lógico automatizado que correlaciona cuatro ejes de decisión en tiempo real: la severidad técnica (NVD), la inteligencia de amenazas (CISA KEV vía MISP), la telemetría de ataques vivos (Wazuh) y un multiplicador de criticidad institucional. Este diseño preserva la soberanía total de los datos mediante orquestación *on-premise* y provee al analista una justificación matemática explícita, resolviendo la fatiga de alertas sin sacrificar la trazabilidad operativa.

4. Definición del Problema

4.1. Planteamiento del Problema

La efectividad de los SOC depende de su capacidad para detectar y responder a incidentes en tiempo real. No obstante, la proliferación de herramientas de defensa aisladas genera "silos de datos". Esta fragmentación dificulta el proceso de triaje, el cual se ha vuelto una tarea exhaustiva debido al alto volumen de datos y la tasa de falsos positivos. Los enfoques de priorización tradicionales dependen del sistema CVSS, el cual es estático y carece de contexto organizacional. La falta de correlación entre alertas internas, criticidad de activos e inteligencia externa impide una gestión eficiente del riesgo.

4.2. Solución Propuesta y Objetivos

Se propone el diseño e implementación de un mecanismo de priorización contextual integrado en una plataforma SOAR. El objetivo general es diseñar e implementar una plataforma SOAR para la correlación y priorización de vulnerabilidades basada en riesgo, mediante la integración de telemetría de SOC y fuentes de inteligencia de amenazas, para optimizar el triaje operativo en un entorno de laboratorio.

Los objetivos específicos son:

1. Definir la arquitectura técnica y los flujos de ingestión de datos (Wazuh y MISP).
2. Diseñar un algoritmo de priorización contextual basado en riesgo (RBVM) combinando CVSS, CISA KEV y criticidad de activos.

3. Desarrollar el motor lógico para la normalización de eventos y el cálculo del riesgo.
4. Implementar una interfaz interactiva (Dashboard) para visualización de triaje.
5. Validar la eficacia del prototipo mediante diseño experimental con datasets de red.

4.3. Importancia del Trabajo

Este trabajo optimiza la eficiencia técnica de los SOC al mitigar la fatiga de alertas. El valor de la propuesta radica en su capacidad de reducir el tiempo medio de respuesta (MTTR) ante amenazas críticas que realmente impactan la operación de la institución (como el Senado), aportando una solución transparente y adaptable frente a los modelos tradicionales de priorización estática.

5. Diseño de la Solución (Arquitectura)

5.1. Arquitectura Funcional por Capas

Para resolver de forma robusta la fragmentación de datos y la dependencia de análisis manuales en el triaje, la solución tecnológica se estructura bajo una arquitectura modular distribuida de cuatro capas funcionales, las cuales eluden el uso de hipervisores locales mediante un esquema híbrido de contenedores efímeros y ejecución nativa. El detalle exhaustivo de los requerimientos y flujos lógicos del sistema se encuentra documentado mediante diagramas UML en el **Anexo A**, mientras que las reglas de detección e integración de los sensores se exponen en el **Anexo D**:

- **Capa de Captura y Telemetría:** Instancias contenerizadas de Wazuh y OpenVAS actúan como sensores primarios. Wazuh recolecta telemetría de eventos de seguridad organizacionales en tiempo real, mientras que OpenVAS aporta el escaneo estático del software para indexar las vulnerabilidades presentes.
- **Capa de Inteligencia de Amenazas (CTI):** Un nodo local encapsulado de la plataforma MISP opera como repositorio táctico interno, sincronizando de manera persistente el catálogo CISA KEV. Esto permite resolver consultas relacionales inmediatas sin penalizar la latencia con peticiones síncronas hacia internet.
- **Capa Lógica de Orquestación (Motor Python):** Constituye el núcleo algorítmico del proyecto. Un servicio desarrollado en Python consume asíncronamente las APIs REST de los sensores y de la plataforma de inteligencia, aislando las variables contextuales requeridas para la priorización ante cada alerta entrante.
- **Capa de Presentación y Persistencia:** Los incidentes enriquecidos se estructuran en documentos NoSQL dentro de MongoDB. Finalmente, un tablero web reactivo en React.js consume estos datos para desplegar el flujo de triaje priorizado.

5.2. Evolución y Modelamiento Matemático de la Fórmula de Riesgo

Siguiendo los fundamentos de la Gestión de Vulnerabilidades Basada en Riesgo (RBVM), el algoritmo implementado no introduce un modelo arbitrario, sino que se deduce a partir de los lineamientos de la norma internacional **ISO/IEC 27005:2011** [10] (Cláusula 8.1 y Anexo E.2.2). Dicho estándar define conceptualmente el riesgo como la combinación de las consecuencias de un evento (impacto) y su probabilidad de ocurrencia (vulnerabilidad y amenaza), lo que se traduce en la Ecuación 1:

$$\text{Riesgo Base} = \text{Severidad (Vulnerabilidad)} \times \text{Impacto del Activo} \tag{1}$$

El paradigma tradicional de los SOC adopta una aproximación incompleta a esta ecuación, asumiendo que la severidad técnica intrínseca provista por la base de datos de la NVD representa el riesgo total (Riesgo $\approx$ CVSS), lo que desencadena la fatiga de alertas al ignorar la topología del entorno institucional. Para corregir esta brecha operativizable, este proyecto evoluciona el modelo base introduciendo factores de ponderación dinámicos.

En primera instancia, se restituye el factor de impacto contextual incorporando la Criticidad del Activo (C_a) como un multiplicador discreto de negocio ($C_a \in [0{,}5,\ 1{,}5]$) asignado al inventario de infraestructura. Esta primera evolución (el Riesgo Estructural) se expresa en la Ecuación 2:

$$R_{\text{contextual}} = \text{CVSS} \times C_a \tag{2}$$

No obstante, un riesgo contextualizado sigue siendo teórico si la falla no presenta vectores de explotación activos en el mundo real. Por ende, se expande la dimensión de la probabilidad (amenaza) mediante la inyección de dos variables lógicas de control deducidas a partir de flujos de datos empíricos:

1. **Explotabilidad Global (I_t):** Un indicador booleano ($I_t \in \{0, 1\}$) extraído mediante la API de MISP que valida si el CVE se encuentra catalogado en el registro de amenazas activas de CISA KEV.

2. **Evidencia Operativa Local (E_o):** Un indicador booleano ($E_o \in \{0, 1\}$) extraído de la telemetría viva de Wazuh que confirma si el activo digital está experimentando vectores de ataque concurrentes relacionados con dicha falla.

Para acotar el índice dentro de una escala estandarizada y auditable para el analista del SOC, se aplica una función de saturación superior configurada en un umbral de corte de 10.0 puntos. Acoplando las constantes de peso analítico asignadas a la peligrosidad de la amenaza externa ($\alpha = 2{,}0$) y la inminencia del ataque perimetral interno ($\beta = 3{,}0$), se obtiene la ecuación final unificada de Riesgo Dinámico (R_f) propuesta en este trabajo (Ecuación 3):

$$R_f = \text{mín}\,(10{,}0,\ (\text{CVSS} \times C_a) + (I_t \times 2{,}0) + (E_o \times 3{,}0)) \tag{3}$$

La asignación de las constantes de ponderación empírica ($\alpha = 2{,}0$ y $\beta = 3{,}0$) fue calibrada específicamente para satisfacer una condición de frontera del modelo: asegurar una penalización contextual máxima de 5.0 puntos. Este umbral garantiza que una vulnerabilidad evaluada estáticamente como "Media"(CVSS = 5.0), al presentar confirmación simultánea de explotación global (CISA KEV) y telemetría de ataque local (Wazuh), escale ineludiblemente al límite de criticidad absoluta (10,0).

La elección de la severidad 5.0 como punto de quiebre se fundamenta en el análisis de incidentes históricos críticos. Vulnerabilidades de alto impacto, como CVE-2014-0160 (*Heartbleed*) [11, 12], fueron categorizadas inicialmente con un CVSS base de 5.0 debido a que estructuralmente solo representaban sobrelecturas de memoria en bibliotecas criptográficas. Sin embargo, su explotación activa permitió el compromiso sistémico de infraestructuras a nivel global mediante la exfiltración de credenciales. Al establecer la penalización máxima en 5.0, el modelo asegura que los vectores de severidad media bajo explotación comprobada no sean invisibilizados por la fatiga de alertas. Adicionalmente, la asimetría en la distribución de los pesos ($\beta > \alpha$) fue definida para reflejar matemáticamente la mayor inminencia de compromiso que representa una intrusión activa en la red interna frente a un indicador de amenaza global.

Es imperativo destacar la decisión arquitectónica de utilizar un modelo de penalización aditiva sobre el riesgo estructural, en lugar de un producto matemático puro. Si las variables de amenaza (I_t y E_o) operaran como multiplicadores, la ausencia temporal de un ataque activo ($E_o = 0$) provocaría el "problema del multiplicador cero", anulando por completo el puntaje e invisibilizando vulnerabilidades de severidad crítica (ej. CVSS 10.0 inactivos). Mediante la suma, el riesgo estructural ($CVSS \times C_a$) se preserva, mientras que las amenazas confirmadas actúan como aceleradores hacia la saturación.

Finalmente, dado que la función de saturación (mín) induce colisiones matemáticas en el límite superior (múltiples alertas alcanzando el tope de 10.0 puntos), el diseño de la plataforma resuelve los empates mediante una lógica de ordenamiento multicriterio (*multilevel sorting*) a nivel de base de datos. Ante una colisión en el Riesgo Final (R_f), el motor de presentación (Dashboard) prioriza el despliegue ordenando de forma descendente, en segunda instancia, por la Evidencia Operativa (E_o) y, en tercera instancia, por la Inteligencia de Amenazas (I_t). Esto garantiza de forma algorítmica que, ante dos incidentes valorados en 10.0, la amenaza con intrusión activa comprobada precederá ineludiblemente a la vulnerabilidad latente en la pantalla del analista.

Para evidenciar la implementación técnica de esta lógica algorítmica, el **Anexo B** expone el diccionario de datos y la estructura JSON de las alertas enriquecidas. Asimismo, el **Anexo C** documenta los tramos de código clave desarrollados en Python responsables de la ingesta de telemetría y la ejecución de la priorización multicriterio en el motor SOAR.

6. Validación de la Solución

6.1. Diseño Experimental y Selección de Datos

Para comprobar empíricamente la viabilidad del modelo propuesto frente a la fatiga de alertas, se estructura un estudio cuantitativo de tipo experimental en un entorno de laboratorio controlado. El diseño busca contrastar la eficacia de priorización del motor RBVM/SOAR frente a los enfoques tradicionales del SOC.

Definición de Líneas Base: El desempeño del algoritmo se comparará contra dos escenarios de control (baselines):

- **Línea Base 1 (Triaje Estático por CVSS):** Representa el estado actual de un SOC sin madurez, donde las alertas se ordenan de mayor a menor únicamente por su severidad técnica.
- **Línea Base 2 (Triaje Contextualizado):** Representa un modelo intermedio que cruza la severidad con el valor del activo ($CVSS \times C_a$), pero carece de la validación de amenaza en tiempo real (I_t y E_o).

Dataset de Experimentación (UNSW-NB15): Para simular el flujo de telemetría y generar las alertas en el SIEM (Wazuh), se inyectará el tráfico de red del dataset público UNSW-NB15 [13]. La elección de este conjunto de datos, en detrimento de alternativas clásicas como KDD99 o NSL-KDD, se fundamenta en su arquitectura moderna. Como demuestran Moustafa y Slay [13], el UNSW-NB15 contiene una topología híbrida que mezcla tráfico benigno contemporáneo con vectores de ataque de baja huella y vulnerabilidades CVE modernas. Esta característica es crucial para el experimento, ya que el ruido de fondo (tráfico normal) generará orgánicamente los falsos positivos necesarios para simular la fatiga de alertas real.

6.2. Métricas de Evaluación

La validación no busca medir la capacidad de *detección* del SIEM, sino la capacidad de *priorización* del orquestador SOAR. Para ello, se seleccionaron dos métricas consolidadas en la literatura de respuesta a incidentes y recuperación de información:

1. Precision@K (Precisión en el Top-K): En la mitigación de la fatiga de alertas, la exactitud global (Accuracy) es una métrica engañosa debido al alto desbalance de clases (el tráfico benigno supera ampliamente a las amenazas reales). Siguiendo el enfoque propuesto por Hassan et al. [14] para el triaje de alertas, se utilizará la métrica $Precision@K$ (con $K = 10$ y $K = 20$). Esta métrica evalúa exclusivamente la calidad de los primeros K resultados mostrados en la pantalla del Dashboard. Si el motor sitúa las alertas críticas reales dentro de estas primeras posiciones visibles, se considera un éxito en la reducción de la carga cognitiva del analista.

2. Reducción del MTTR (Tiempo Medio de Respuesta): El MTTR es la métrica de eficiencia operativa estándar definida por el marco de respuesta a incidentes del NIST (SP 800-61) [15]. Para efectos del laboratorio, se realizará un cálculo de MTTR simulado. Se medirá matemáticamente cuántos registros falsos o de baja prioridad tendría que haber auditado manualmente un analista en la Línea Base 1 y 2 antes de encontrar el verdadero positivo, en comparación con la posición en la que el modelo RBVM despliega dicha alerta. Esta diferencia de posiciones se traducirá en horas-hombre ahorradas, validando el impacto del sistema en el negocio.

7. Limitaciones

El diseño e implementación de esta plataforma contempla las siguientes restricciones de alcance técnico:

- **Entorno Simulado e Infraestructura Efímera:** Debido a limitaciones de *hardware* para virtualización pesada tipo hipervisor, el sistema operará mediante ejecución nativa y despliegues efímeros en contenedores Docker (*mocking* de APIs). La validación matemática se realizará sobre *datasets* inyectados localmente, excluyendo el despliegue directo en la topología de red productiva de la institución.
- **Alcance de la Orquestación Automatizada:** Las capacidades SOAR de esta iteración se restringen estrictamente a las fases de ingesta, correlación, enriquecimiento táctico y priorización matemática en el *Dashboard*. No se contempla la ejecución de acciones de respuesta automática intrusiva (tales como modificaciones de reglas de *firewall* o parcheo desatendido) para evitar disrupciones accidentales en la disponibilidad de los servicios críticos del cliente.

Agradecimientos

El autor agradece a la Escuela de Ingeniería Informática de la Universidad de Valparaíso y al profesor guía Cristian Carrión por el apoyo brindado en la formulación de este proyecto.

Aspectos éticos

Confirmamos que el desarrollo de este trabajo se enmarca en un entorno de laboratorio aislado utilizando bases de datos públicas (datasets), por lo cual no compromete información sensible ni afecta infraestructuras productivas reales, alineándose con las normativas éticas de ciberseguridad.

Declaración de autoría

Jorge Eduardo Álvarez Alarcón: Conceptualización, Metodología, Software, Validación, Investigación, Redacción de borrador original y edición.

Declaración de intereses

El autor declara no tener conflictos de intereses financieros, personales ni institucionales que pudieran influir en los resultados o interpretaciones del presente trabajo.

Referencias

[1] S. Tariq, M. B. Chhetri, S. Nepal, C. Paris, Alert fatigue in security operations centres: Research challenges and opportunities, ACM Computing Surveys (2025). doi:10.1145/3723158.

[2] Anonymous, Exploration of open source siem tools and deployment of an appropriate wazuh-based solution for strengthening cyberdefense, in: IEEE Xplore, 2024, pp. 1–8. doi:10.1109/10708054/10708053/10708476.

[3] B. D. Jaiswal, Designing scalable software automation frameworks for cybersecurity threat detection and response, International Journal of Scientific Research and Management (IJSRM) 13 (02) (2025) 1958–1980. doi:10.18535/ijsrm/v13i02.ec03.

[4] Taylor, Francis, Vulnerability scoring metric of cvss needs to be adjusted per each product, Information Security Journal: A Global Perspective (2025).

[5] M. Rieppi, Vulnerability management: A risk-based optimization, Master's thesis, Università degli Studi di Padova, Dipartimento di Matematica "Tullio Levi-Civita"(2025).
URL https://hdl.handle.net/20.500.12608/101996

[6] Anonymous, Explainable risk-based vulnerability prioritization in hybrid cloud: Integrating cvss, epss, and cisa kev with asset criticality, World Journal of Advanced Research and Reviews (2026).
URL https://wjarr.com/sites/default/files/fulltext_pdf/WJARR-2026-1006.pdf

[7] A. Abakar, et al., A cyber threat intelligence model using misp and machine learning in a soc environment, International Journal of Advanced and Applied SciencesOpen Access (2025).
URL https://www.science-gate.com/IJAAS/Articles/2025/2025-12-11/1021833ijaas202511001.pdf

[8] R. Kits, et al., What's in a cyber threat intelligence sharing platform?: A mixed-methods user experience investigation of misp, in: Proceedings of the ACM, 2021, pp. 1–12. doi:10.1145/3485832.3488030.

[9] runZero, Kevology: An analysis of exploits, scores, and timelines on the cisa kev, Tech. rep., runZero (2024).
URL https://www.runzero.com/uploads/documents/reports/KEVology-An-analysis-of-exploits-scores-and-timelines-on-the-CISA-KEV.pdf

[10] International Organization for Standardization, Information technology — Security techniques — Information security risk management, Standard, International Organization for Standardization (Jun. 2011).

[11] National Vulnerability Database (NVD), CVE-2014-0160 Detail, https://nvd.nist.gov/vuln/detail/CVE-2014-0160, accedido en mayo de 2026 (2014).

[12] Z. Durumeric, J. Kasten, D. Adrian, J. A. Halderman, M. Bailey, F. Li, N. Weaver, J. Amann, J. Beekhof, V. Paxon, et al., The matter of heartbleed, in: Proceedings of the 2014 Conference on Internet Measurement Conference, 2014, pp. 475–488.

[13] N. Moustafa, J. Slay, Unsw-nb15: a comprehensive data set for network intrusion detection systems, in: 2015 Military Communications and Information Systems Conference (MilCIS), IEEE, 2015, pp. 1–6.

[14] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, A. Bates, Nodoze: Combatting threat alert fatigue with automated provenance triage, in: Network and Distributed System Security Symposium (NDSS), 2019, pp. 1–15.

[15] P. Cichonski, T. Millar, T. Grance, K. Scarfone, Computer Security Incident Handling Guide, Special Publication (SP) 800-61 Rev. 2, National Institute of Standards and Technology (NIST) (August 2012).

Anexo A. Diagramas de Ingeniería de Software (UML)

Anexo A.1. Catálogo de Requerimientos

ID	Descripción del Requerimiento	Tipo
RF-01	El motor lógico debe conectarse vía API REST al SIEM (Wazuh) para extraer alertas de seguridad concurrentes (E_o).	Funcional
RF-02	El sistema debe consultar de forma local la plataforma MISP para verificar la presencia de un CVE en el catálogo CISA KEV (I_t).	Funcional
RF-03	El sistema debe calcular el índice dinámico RBVM combinando severidad, inteligencia, telemetría y criticidad.	Funcional
RF-04	La interfaz (Dashboard) debe presentar las alertas ordenadas de forma estrictamente descendente según el Riesgo Final (R_f).	Funcional
RNF-01	**Soberanía de Datos:** El procesamiento de telemetría debe realizarse de forma local (*on-premise*), sin enviar datos a nubes públicas.	No Funcional (Seguridad)
RNF-02	**Tecnología:** El motor de orquestación central debe ser desarrollado íntegramente en lenguaje Python 3.x.	No Funcional (Arquitectura)

Tabla A.2: Catálogo de requerimientos principales del sistema SOAR.

Anexo A.2. Diagrama de Casos de Uso

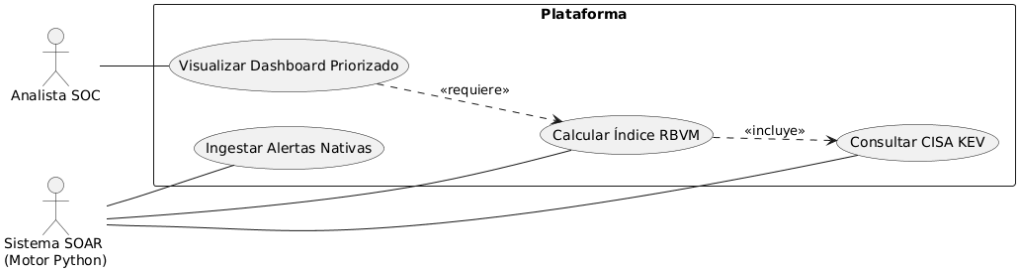


Figura A.1: Diagrama de Casos de Uso: Interacción entre el analista y el orquestador SOAR.

Especificación de Caso de Uso: CU-01	
Nombre	Calcular Índice RBVM y Priorizar Alerta
Actores	Sistema SOAR (Principal), Analista SOC (Secundario)
Propósito	Calcular dinámicamente el riesgo de una alerta integrando telemetría local, inteligencia de amenazas externa y criticidad del activo para ordenar el flujo de triaje.
Tipo	Primario, Automatizado.
Descripción	El sistema captura una alerta bruta, extrae su CVE, consulta su estado de explotación global, verifica la criticidad del activo afectado y aplica la fórmula matemática RBVM.
Referencias Cruzadas	RF-01, RF-02, RF-03, RF-04
Curso Normal de Eventos	1. El Sistema SOAR detecta e ingesta una alerta bruta desde Wazuh. 2. El motor extrae los metadatos clave (IP origen, identificador CVE). 3. Consulta la base de datos local para obtener la Criticidad del Activo (C_a). 4. Envía petición a la API de MISP para validar si el CVE está en CISA KEV (I_t). 5. El algoritmo procesa las variables e imputa el Riesgo Final (R_f). 6. El registro JSON enriquecido se persiste en MongoDB. 7. La alerta se publica en el Dashboard ordenada de forma descendente.

Tabla A.3: Especificación extendida del Caso de Uso principal del orquestador.

Anexo A.3. Diagramas de Secuencia y Estado

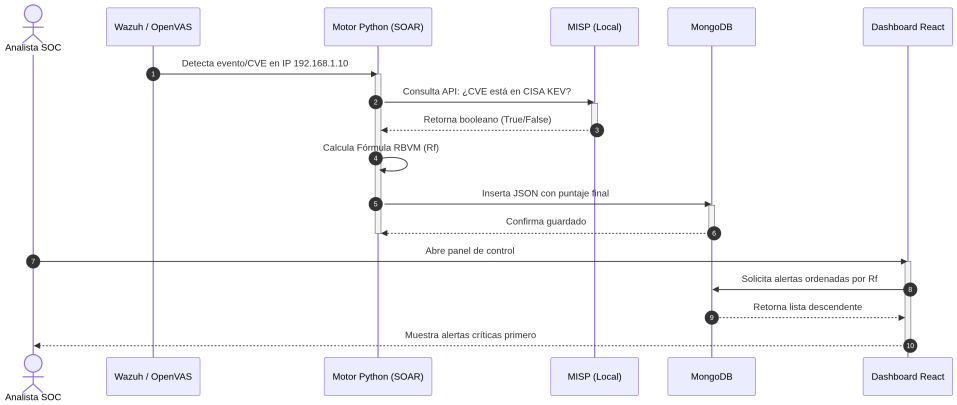


Figura A.2: Diagrama de Secuencia: Flujo automatizado de triaje y priorización RBVM.

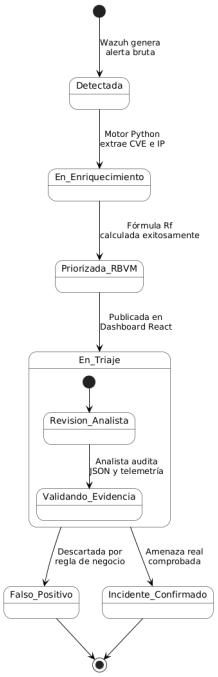


Figura A.3: Diagrama de Estado: Ciclo de vida de una alerta en la plataforma.

Anexo B. Modelo de Datos y Especificaciones JSON

Dado el carácter no relacional y flexible de las alertas de ciberseguridad, se adopta MongoDB como motor de almacenamiento. A continuación, se presenta el esquema JSON resultante de la correlación automatizada, el cual incluye las variables del cálculo de riesgo (RBVM):


```
{
  "_id": "64f1a2b3c9...",
  "timestamp_ingesta": "2026-05-25T14:30:00Z",
  "activo": {
    "ip_origen": "192.168.1.10",
    "rol_negocio": "Servidor Base de Datos - Finanzas",
    "multiplicador_criticidad_Ca": 1.5
  },
  "indicadores_tecnicos": {
    "cve_id": "CVE-2021-44228",
    "score_base_cvss": 10.0,
    "software_afectado": "Apache Log4j"
  },
  "inteligencia_amenazas": {
    "en_cisa_kev_It": true,
    "origen_inteligencia": "MISP Local Node"
  },
  "telemetria_operativa": {
    "alerta_activa_wazuh_Eo": true,
    "tipo_alerta": "Intentos de inyección JNDI múltiples"
  },
  "calculo_rbvm": {
    "formula_applied": "min(10.0, (10.0 * 1.5) + (1 * 2.0) + (1 * 3.0))",
    "riesgo_final_Rf": 10.0,
    "nivel_prioridad": "CRÍTICO INMINENTE"
  }
}
```

Anexo C. Diseño Algorítmico del Motor de Priorización

A continuación, se detalla el pseudocódigo que define la lógica de la función de cálculo de riesgo y el ordenamiento de la base de datos:

```
INICIO FUNCION calcular_riesgo_dinamico(cvss, ca, it, eo)
    // Convertir booleanos de amenaza a enteros
    SI it ES VERDADERO ENTONCES val_it = 1 SINO val_it = 0
    SI eo ES VERDADERO ENTONCES val_eo = 1 SINO val_eo = 0

    // Calcular riesgo base estructural
    riesgo_base = cvss * ca

    // Aplicar modelo aditivo
    riesgo_calculado = riesgo_base + (val_it * 2.0) + (val_eo * 3.0)

    // Aplicar umbral de saturación
    riesgo_final = MINIMO(10.0, riesgo_calculado)

    RETORNAR riesgo_final
FIN FUNCION
```

Anexo D. Reglas de Detección e Integración (Sensores)

El motor de orquestación interactúa con las plataformas a través de sus interfaces RESTful. Las integraciones principales incluyen:

- **Wazuh API (Puerto 55000):** Se utiliza el endpoint `/security/user/authenticate` para la obtención del token JWT. Posteriormente, se consulta el endpoint `/alerts` mediante peticiones GET, aplicando filtros por ID de agente y ventana temporal para extraer la evidencia operativa (E_o).
- **MISP API:** Mediante la librería `PyMISP`, se envía una petición POST al endpoint `/attributes/restSearch`. El cuerpo (*payload*) de la petición incluye el identificador CVE como llave de búsqueda para verificar si el atributo existe dentro del *feed* de CISA KEV sincronizado localmente (I_t).

A continuación, se detalla un ejemplo empírico de regla de decodificación personalizada en el motor de **Wazuh** (formato XML), diseñada para activar la variable de Evidencia Operativa ($E_o = 1$) frente a patrones de inyección anómalos, correlacionando con el flujo del Anexo B:

```
<group name="web, attack, sqli, jndi">
  <rule id="100050" level="12">
    <if_sid>31100</if_sid>
    <match>jndi:ldap:|jndi:rmi:|jndi:dns:</match>
    <description>SOAR-Trigger: Intento de Inyección JNDI (Posible Log4j/RCE)
    detectado en payload HTTP.</description>
    <mitre>
      <id>T1190</id>
    </mitre>
    <group>web_attack,pci_dss_11.4,gdpr_IV_35.7.d,</group>
  </rule>
</group>
```